

Code changes from implicit to CEL

fe-large-displacement

Five formulations of one strip-indenter problem. Geometry, mesh, material and contact are built once. Each model script applies a delta, writes the deck, checks it, and submits. What follows is that shared core, a one-page map of the deltas, then only the code that changes.

1 Shared core

Builder: `examples/common/indenter_model.py:99 build(P, model_name, solver)`. Soil block, rigid strip indenter, C3D8R, rollers, encastre base, general contact. Default material is elastic on purpose.

The step, the drive and the contact helper are the first split.

```

191 # -- step -----
192 step_name = 'Push'
193 if solver == 'explicit':
194     time_period = P.ind_depth / float(P.ind_vel)
195     steps.explicit_step(model, step_name, 'Initial', time_period,
196                         nlgeom=True)
197     steps.smooth_ramp(model, 'Ramp', P.ramp_frac, time_period)
198 else:
199     time_period = 1.0
200     steps.implicit_step(model, step_name, 'Initial', time_period,
201                         nlgeom=True)
202     model.TabularAmplitude(name='Ramp', timeSpan=STEP,
203                           data=((0.0, 0.0), (time_period, 1.0)))
204 steps.energy_output(model, step_name, interval=200)
205 model.HistoryOutputRequest(
206     name='RP', createStepName=step_name,
207     region=asm.sets['IndRP'],
208     variables=('U1', 'U2', 'U3', 'RF1', 'RF2', 'RF3'),
209     numIntervals=500)
210 model.fieldOutputRequests['F-Output-1'].setValues(
211     variables=('S', 'PE', 'PEEQ', 'LE', 'U', 'V', 'A', 'RF', 'CSTRESS',
212              'EVF', 'SDV', 'STATUS'),
213     numIntervals=20)
214
215 # -- boundary conditions -----
216 _soil_bcs(model, asm, soil_inst, P)
217 if solver == 'explicit':
218     # A prescribed VELOCITY is the natural drive for an explicit run: it
219     # gives a constant penetration rate, which is what makes the resistance
220     # curve interpretable, and the SMOOTH STEP ramp keeps the first
221     # increment from sending a stress wave through the whole model.
222     model.VelocityBC(name='Push', createStepName=step_name,
223                     region=asm.sets['IndRP'],
224                     v1=0.0, v2=0.0, v3=-P.ind_vel,
225                     vr1=0.0, vr2=0.0, vr3=0.0,
226                     amplitude='Ramp', distributionType=UNIFORM)
227 else:
228     model.DisplacementBC(name='Push', createStepName=step_name,
229                         region=asm.sets['IndRP'],
230                         u1=0.0, u2=0.0, u3=-P.ind_depth,

```

```

231         ur1=0.0, ur2=0.0, ur3=0.0,
232         amplitude='Ramp', distributionType=UNIFORM)
233
234     # -- contact -----
235     contact.friction_property(model, 'Interface', mu=P.friction, hard=True)
236     if solver == 'explicit':
237         contact.general_contact(model, step_name, 'Interface')
238     else:
239         contact.general_contact_std(model, step_name, 'Interface')

```

`solver='implicit'` selects the Standard element library, a `*Static` step, a displacement BC and Standard general contact. `solver='explicit'` selects Explicit elements, `*Dynamic, Explicit`, a velocity BC and Explicit general contact. Everything above that call is shared.

2 Delta map

Rung	Step / job	Extra block	Check
Implicit	<code>*Static</code> <code>lib/fldlib/steps.py:42</code> <code>lib/fldlib/jobs.py:78</code>	none	no <code>*Adaptive Mesh</code>
Implicit ALE	same Static step	<code>*Adaptive Mesh</code> on Standard, <code>initial_mesh_sweeps=None</code> <code>lib/fldlib/ale.py:221</code>	diagnostics with <code>include_static=True</code>
Explicit	<code>*Dynamic, Explicit</code> <code>lib/fldlib/steps.py:66</code> <code>lib/fldlib/jobs.py:39</code>	optional <code>*Section Controls</code>	ALLKE/ALLIE
Explicit ALE	same Explicit step	<code>*Adaptive Mesh</code> plus <code>*Diagnostics</code> <code>lib/fldlib/inpedit.py:137</code>	<code>assert_active</code> on the .msg
Explicit CEL	same Explicit step	Eulerian part, EC3D8R, void, volume fraction <code>lib/fldlib/cel.py:43</code>	void fraction, no ALE

3 Implicit

```

42 def implicit_step(model, name, previous='Initial', time_period=1.0,
43                  nlgeom=True, description=''):
44     """A geometrically nonlinear *Static step with automatic stabilisation.
45
46     Stabilisation is a fictitious viscous force. It is on because a
47     penetration analysis in Standard otherwise stops converging as soon as
48     the soil starts to fail. Check ALLSD against ALLIE afterwards. The
49     implicit models in this repository exist to show where Standard gives
50     up, not to hide that it did.
51     """
52     from abaqusConstants import ON, OFF, DISSIPATED_ENERGY_FRACTION
53     return model.StaticStep(
54         name=name, previous=previous, description=description,
55         nlgeom=ON if nlgeom else OFF,
56         timePeriod=time_period,
57         initialInc=0.005, minInc=1.0e-8, maxInc=0.05,
58         maxNumInc=1000,
59         stabilizationMethod=DISSIPATED_ENERGY_FRACTION,
60         stabilizationMagnitude=2.0e-4,
61         adaptiveDampingRatio=0.05,
62         continueDampingFactors=ON,
63     )

```

Stabilisation is a fictitious viscous force. Check ALLSD against ALLIE. The job helper is `lib/fldlib/jobs.py:78`. The model script forbids ALE in the deck.

```

44     job = jobs.make_implicit_job('Imp_' + case, job_name, n_cpus=n_cpus,
45                                 description=CASE_DOC[case])
46     inp = jobs.write_input(job, workdir)
47
48     chk = report.InpCheck(inp)
49     chk.requires(r'^\*Step,.*nlgeom=YES', 'the step is geometrically
nonlinear')
50     chk.requires(r'^\*Static', 'static procedure')
51     chk.forbids(r'^\*Adaptive Mesh,', 'no ALE -- purely Lagrangian')
52     chk.report()

```

Expect abort on Too many attempts made for this increment. That is the result.

4 Implicit ALE

Same `build(..., solver='implicit')` call. Then attach a restricted domain. Standard has no initial mesh sweeps, so they are omitted.

```

51     ale.make_controls(model, 'ALE_BC', smoothing=UNIFORM)
52     box = indenter_model.ale_box(P)
53     n_adaptive, bbox = ale.box_element_set(asm, handles['soil_inst'],
54                                           'ALE_Box', box)
55     ale.describe_box_set('ALE_Box', n_adaptive, bbox, total=n_soil)
56     ale.add_domain(model, handles['step_name'], asm.sets['ALE_Box'], 'ALE_BC',
57                  frequency=P.ale_frequency, mesh_sweeps=P.ale_sweeps,
58                  initial_mesh_sweeps=None)
59
60     n_cpus = int(os.environ.get('FLD_NCPU', P.n_cpus))
61     job = jobs.make_implicit_job('ImpAle_' + case, job_name, n_cpus=n_cpus,
62                                 description=CASE_DOC[case])
63     inp = jobs.write_input(job, workdir)
64     inpedit.inject_diagnostics(inp, mode='summary', include_static=True)

```

`add_domain` is shared with Explicit. The Standard path is the `None`:

```

242     kwargs = dict(
243         region=region,
244         controls=controls,
245         frequency=frequency,
246         meshSweeps=mesh_sweeps,
247     )
248     if initial_mesh_sweeps is not None:
249         kwargs['initialMeshSweeps'] = initial_mesh_sweeps
250     return step.AdaptiveMeshDomain(**kwargs)

```

Abaqus/Standard ALE is Lagrangian-only, one smoothing algorithm, intended for wear and acoustics. Do not expect skirt penetration from it. The model exists so the keywords are visible. `assert_active(..., required=False)` because Standard may omit the Explicit-style `.msg` block.

5 Explicit

```

66 def explicit_step(model, name, previous, time_period, nlgeom=True,
67                  improved_dt=True, description=''):
68     """A geometrically nonlinear '*Dynamic, Explicit' step.
69
70     'nlgeom' is ON by default and you should essentially never turn it off in

```

```

71     this repository: without it there is no large displacement, the ALE
72     formulation is meaningless, and contact will behave strangely.
73
74     'improved_dt' ('improvedDtMethod=ON') gives a less conservative stable
75     increment estimate for hex elements. It is the CAE default and typically
76     buys 10-40 % on run time for free.
77     """
78     from abaqusConstants import ON, OFF
79     return model.ExplicitDynamicsStep(
80         name=name, previous=previous, description=description,
81         timePeriod=time_period,
82         nlgeom=ON if nlgeom else OFF,
83         improvedDtMethod=ON if improved_dt else OFF,
84     )

```

Job: lib/fldlib/jobs.py:39. DOUBLE_PLUS_PACK is not optional for soil. Drive is velocity with a SMOOTH STEP ramp (examples/common/indenter_model.py:222).

Cases in examples/indenter/explicit:

- A — baseline. Distortion abort. Compare depth with implicit.
- B — four times the velocity. Energy test. See docs/01-explicit-quasi-static.md.
- C — distortion control injected after *Preprint. See docs/04-section-controls.md. Do not combine it with enhanced hourglass.

fldlib.inpedit.section_controls_keyword raises if you ask for both.

6 Explicit ALE

Same Explicit build. The only new block is the adaptive domain, on a restricted element set.

```

63     domain_elset = None
64     if case != 'A':
65         ale.make_controls(model, 'ALE_BC', smoothing=UNIFORM)
66         box = indenter_model.ale_box(P)
67         n_adaptive, bbox = ale.box_element_set(
68             asm, handles['soil_inst'], 'ALE_Box', box)
69         ale.describe_box_set('ALE_Box', n_adaptive, bbox, total=n_soil)
70         domain_elset = 'ALE_Box'
71         ale.add_domain(model, handles['step_name'], asm.sets['ALE_Box'],
72                       'ALE_BC', frequency=P.ale_frequency,
73                       mesh_sweeps=P.ale_sweeps,
74                       initial_mesh_sweeps=P.ale_initial_sweeps)

```

The box helper is examples/common/indenter_model.py:323. An empty box is an inert domain. CAE supports one domain per step and silently replaces a second call. add_domain raises instead.

CAE cannot write *Diagnostics. Inject it after the procedure card.

```

137 def inject_diagnostics(inp_path, mode='summary', include_static=False):
138     """Insert '*Diagnostics, adaptive mesh=<mode>' after each matching
139     procedure card.
140
141     Without this, the '.msg' carries only an end-of-step summary and an ALE
142     domain that never swept looks identical to one that swept perfectly. With
143     it, Abaqus/Explicit writes a per-adaptive-increment block reporting mesh
144     sweeps and the percentage of nodes moved.
145
146     'include_static=True' also attaches the keyword after '*Static', which is
147     the Abaqus/Standard ALE path. Standard may still omit the Explicit-style
148     diagnostic block. The injection is then a no-harm request, not proof.

```

Then `ale.assert_active(msg)`. Case A (no ALE) must report no activity, or the parser is wrong. Measured keyword defaults, the MESH SWEEPS documentation error, and general-contact compatibility are in `docs/02-ale-adaptive-meshing.md`. Several well-separated regions and domain-level parallelisation are in `docs/03-ale-multi-region-parallel.md`. That study is not a separate model. The caisson script uses the two-region pattern.

7 Explicit CEL

The soil is no longer the shared Lagrangian part. It is an Eulerian box with empty space above the surface. The indenter stays Lagrangian. Element type EC3D8R. No `*Geostatic`.

```

43 def eulerian_section(model, name, material, instance_key=None):
44     """Create an '*Eulerian Section'.
45
46     The 'data' argument is a dict mapping the **Eulerian material instance
47     name** to the **material name**. The instance name is an arbitrary label
48     you
49     choose; it is the handle that 'MaterialAssignment' and the EVF output
50     variables use. The convention '<material>-1' lowercased is what CAE
51     generates interactively, and the examples here follow it so that decks
52     written by hand and by CAE look the same.
53
54     Returns '(section, instance_key)'.
55     """
56     key = instance_key or (material.lower().replace(' ', '_') + '-1')
57     section = model.EulerianSection(name=name, data={key: material})
58     return section, key

```

Case A of `examples/indenter/explicit_cel` sets void thickness to zero (the silent mistake). Case B sets it to half the penetration depth. After assignment, `cel.check_void_fraction` warns if too little of the domain is empty. The deck must contain `*Eulerian Section`, `EC3D8R`, `*Initial Conditions`, `type=VOLUME FRACTION` and `*Contact`, and must not contain `*Adaptive Mesh`.

8 Checks that apply on more than one rung

Check	Where
InpCheck on the written .inp	every model
ALLKE/ALLIE, ALLAE, ETOTAL	every Explicit run
*Diagnostics then assert_active	both ALE models
check_void_fraction	CEL
tools/check_odb_finite.py	any user-material job

Write the input. Read it. Assert. Then submit.